OPERATIONS & MAINTENANCE DOCUMENTATION

Facility: Adventure Bakery

Host Infrastructure: Hostinger VPS (srv1472620)

Database Backend: Supabase (PostgreSQL)

Service Architecture: Permanent Python MQTT Daemon

Last revised: 2026-10-12 (walk-in freezer out of service; page guide added)

Source: `sop-drafts/yolink_operations_guide.html` in the repository

System Overview

This deployment establishes a permanent, secure data-collection pipeline mapping telemetry from physical YoLink wireless environment sensors directly into a central Supabase relational table. An asynchronous background process runs under systemd supervision on a dedicated Hostinger Virtual Private Server. The captured readings power the Team Portal **Temperature Monitoring** report (Compliance → Temperature Logs).

The system has **three parts**, and this document covers all of them:

1. **Ingestion** — the VPS daemon that receives sensor telemetry and writes it to `temperature_logs`. Everything up to *Automatic Alerting* below.
2. **Alerting** — a scheduled check that reads those rows every fifteen minutes, judges each in-service unit against a written limit, and emails somebody when a unit is out of range, has stopped reporting, or has a flat battery. See *Automatic Alerting & Escalation*.
3. **The Temperature Monitoring page** in the Team Portal — where the readings are read, alerts are answered, limits and service state are set, and the monthly review is started. See *Using the Temperature Monitoring page*: that section is written for the person at the screen, not the person at the server.

Collecting readings is not the same as monitoring them. Until the alerting was added, a walk-in could sit warm — or a sensor could stop reporting entirely — and nothing would say so until somebody happened to open the report. In the first ten weeks of logging, transmission stopped three times, once for more than three days, and nobody noticed.

Source Code & Repository

The logger source is version-controlled in the Adventure Bakery Team App repository. The copy running on the VPS should always match what is in git — treat git as the source of truth and deploy changes from there.

Item	Location
GitHub repository	`github.com/Adv-bakes/Adventure-Bakery-Team-App`
Path within repo	`vps/yolink-logger/`
Tracked files	`main.py` • `requirements.txt` • `.env.example` • `.gitignore` • `README.md`
Live copy on VPS	`/root/yolink-logger/` (running under systemd)

Important: the real `.env` (YoLink UAID/Secret, Supabase service-role key) is **never** committed to git — it exists only on the VPS. The repo ships a `.env.example` listing the required variable names, and the repo `README.md` has full setup and deploy steps.

Deploy an updated main.py from git to the server

```
# Option A - copy the file straight up
scp vps/yolink-logger/main.py root@srv1472620:~/yolink-logger/main.py
ssh root@srv1472620 'sudo systemctl restart yolink-logger.service'

# Option B - if the VPS folder is itself a git clone
cd ~/yolink-logger && git pull && sudo systemctl restart yolink-logger.service
```

Hardware & Database Mapping

The Python application maps arriving telemetry packets from hex-encoded physical hardware IDs to human-readable names (via `DEVICE_MAPPING` in `main.py`) before validating parameters and inserting rows. Update that dictionary when a sensor is added or replaced.

Device Hardware ID	Mapped Room Label	Database Assignment
`d88b4c010010b5da`	Walk-In Refrigerator	`public.temperature_logs`
`d88b4c010010b513`	Walk-In Freezer	`public.temperature_logs`
`d88b4c010010b70a`	Bakery Floor	`public.temperature_logs`

> **The mapped room label is a join key — do not rename it casually**
>
> Each name in `DEVICE_MAPPING` is written into `temperature_logs.equipment_name`, and the alerting matches limits to readings **by that exact string** (`temperature_limits.equipment_name`). There is no foreign key and no constraint tying the two together.
>
> So if you rename a unit in `main.py` — or add a sensor without adding its limits row — readings keep arriving and the report keeps drawing them, but **the alerting silently stops covering that unit**. Nothing errors; the run simply reports fewer units checked. **Change both sides together**, then confirm with the query in troubleshooting item F.

Operational Architecture Deployment

1. Project Directory & Environment Structure

```
/root/yolink-logger/
|-- venv/           isolated Python deps (paho-mqtt v2, supabase-py)
|-- .env            protected credentials (NOT in git)
`-- main.py       execution core script
```

2. Linux Background Daemon Integration (systemd)

Defined at `/etc/systemd/system/yolink-logger.service`. `Restart=always` revives the process after crashes or reboots.

```
[Unit]
Description=YoLink MQTT Temperature Logger to Supabase
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
WorkingDirectory=/root/yolink-logger
```

```
ExecStart=/root/yolink-logger/venv/bin/python main.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Automatic Alerting & Escalation

Everything above gets readings into the database. This section is what turns those readings into *monitoring*: a scheduled job judges each in-service unit against a written limit and tells somebody when it fails.

Components

Component	What it is
`public.temperature_limits`	One row per sensor: the limit, whether the unit is in service, how long a silence is tolerated (`stale_hours`), and who gets emailed. Edited in the app — see below.
`public.temperature_alerts`	One row per excursion, opened and closed automatically. **Append-only:** no client may INSERT, UPDATE or DELETE. The only writable part is the acknowledgement, through a database function.
`temperature-alert`	Supabase Edge Function holding the logic. Deploy with `supabase functions deploy temperature-alert` . Needs `RESEND_API_KEY` in the project's Edge Function secrets to send mail.
`temperature-alert-check`	`pg_cron` job on `*/15 * * * *` that calls the function. Inspect with `select * from cron.job;` , history in `cron.job_run_details` .

What raises an alert

Kind	Condition
`out_of_range`	The **two most recent** readings are both beyond the unit's limit. Two rather than one, so a door held open while ingredients are carried out does not page anyone. A reading exactly *at* the limit passes; beyond it fails.
`no_data`	Nothing logged for longer than the unit's `stale_hours` (default 6). **This is the one that matters most.** A dead sensor reads as perfect compliance, so an alert that fired only on a bad reading would have stayed silent through all three historical logging gaps.
`low_battery`	The sensor reports a low battery, or level 1 of 4 — which is how `no_data` usually begins.

Behaviour worth knowing before you debug it

- **One open alert per unit per kind.** Enforced by a partial unique index in the database rather than by the function, so a unit sitting warm for six hours produces one alert and one email — not twenty-four — and two overlapping runs cannot both open one.
- **Unacknowledged alerts are re-sent every 24 hours** until somebody responds, so one missed email does not mean a warm walk-in goes unnoticed indefinitely.
- **A stale sensor is not also judged against its limit.** Its last reading may be three days old; treating that as current would raise a second, misleading alert and send the responder to the wrong problem.
- **Out-of-service units and the Bakery Floor raise nothing.** The Bakery Floor is ambient monitoring, not a storage unit, and has no limit by design. A unit is placed out of service when it is switched off, not when it merely happens to be empty.
- **Taking a unit out of service also closes whatever it had open.** At the next run, any alert still open for that unit is closed with the reason *Unit placed out of service* — no “Resolved” email, because the unit is not within limits, it is switched off — and its card on the Notifications page is resolved. Added 2026-10-12, when the walk-in freezer was switched off with its out-of-range alert open: before this the job simply stopped looking at the unit, so the alert would have stayed open for good, and the one-open-alert-per-unit rule would then have blocked a fresh alert after a return to service.
- **Alerts close themselves** when readings return within limits, and send a short "Resolved" email.

Configuring limits and recipients

In the Team Portal: **Compliance** → **Temperature Monitoring** → **Temperature Alerts** → **Limits**. The editor is visible to **admin and owner accounts only**; staff see the limits as a read-only table, deliberately — a limit is what compliance *means*, so changing one is not a floor-level action. Each storage unit carries its own limit, staleness tolerance, recipient list and **In service** switch; moving the switch asks for a reason, which is recorded (SOP-401 Part 7). Field by field, and what each one does, is in *Using the Temperature Monitoring page* below.

Leaving the recipient list blank falls back to every admin and owner account's email address. That default exists so alerting works from the moment it is deployed rather than after somebody remembers to fill in a column — but it tracks whoever currently holds an admin role, so it drifts: grant somebody admin next year for an unrelated reason and they begin receiving walk-in temperature alerts. **Set recipients explicitly per unit** once the system is in routine use.

Testing tip: before deliberately tripping an alert, pin the unit's recipients to a single address — and leave the pin in place until the alert has *cleared*. The resolution email uses whatever the list says at the moment it closes, not what it said when it opened.

Responding to an alert

Open the alert on the Temperature Monitoring page and **acknowledge it with what you found and what you did**. That sentence is the corrective-action record; it is written once and **cannot be edited afterwards**, so record the actual time and the probe reading. An alert nobody acknowledged is only a notification — the acknowledgement is what makes it evidence.

The procedure governing the response, the limits and the monthly review is **SOP-401 Temperature-Controlled Storage**, recorded on **FRM-401 Temperature Monitoring Review**. (Both issued and in force since 2026-09-08; SOP-401 is at v4, 2026-10-12.) Product whose safety is in doubt after an excursion is placed on Hold under **FSQM-018** and recorded on **FRM-702** — that decision belongs to the SQF Practitioner, not to whoever happens to answer the email.

Using the Temperature Monitoring page

Everything above is for whoever maintains the server and the alerting. This section is for whoever *uses* the page: what each part of it is, what each button does, and what is recorded when it is pressed. The page is **Compliance → Temperature Monitoring** in the Team Portal (`/team/compliance/temperature`). Everyone with a team account can open it; two things on it are for admin and owner accounts only, and are marked below.

The page, top to bottom

What you see	What it is and what it does
Daily / Weekly / Monthly tabs, and **From / To** dates	The window the page reads. The tabs set it to the last 24 hours, 7 days or 30 days; typing dates in From and To overrides them. Everything below the controls — summary, battery, chart, exports — follows this window. The window is *rolling*: "Monthly" means the last thirty days, not the calendar month, which is why the monthly review has its own month picker (below).
An amber **stale-data** banner	Shown when the newest reading is more than six hours old, or when the window holds no readings at all. It links to this guide. It means the sensors or the daemon have stopped, not that a unit is warm; a `no_data` alert will have been raised for each in-service unit (see *Automatic Alerting*).
Start FRM-401 Review (gold button)	Opens a small month picker and starts, or continues, that month's FRM-401 Temperature Monitoring Review with the figures filled in. See *The monthly review* below. Staff, admin and owner accounts see it.
Export → PDF report / Spreadsheet (CSV)	The window's readings as a file. The PDF carries the per-unit summary; the CSV carries every reading with its timestamp, timezone, unit, device id, both temperatures, humidity and battery columns. This is the export an auditor is handed when they ask for the temperature record.
Temperature Alerts card	The part of the page SQF 11.6.2.3 asks for: what the limits are and what was done when a reading broke one. Described piece by piece in the next table.
Summary by Equipment	One row per sensor for the window: number of readings, minimum, maximum and average °F, average humidity. **Click a row** to open the drill-down: every individual reading for that unit in the window, with its timestamp, and its own Export (PDF or CSV). The Avg Humidity column always reads 0 — the sensors do not report humidity.
Battery Status	A gauge per sensor of its last reported battery level, 1 (low) to 4 (full). A red banner lists any sensor reporting a low battery or level 1 — replace that battery before the sensor stops reporting (troubleshooting item D).
Average Temperature °F chart	Average °F per time bucket, one line per sensor, over the window. A picture of the same readings the summary counts; useful for seeing a door left open or a defrost cycle.

The Temperature Alerts card

What you see	What it is and what it does
A green line, *No open alerts*	Every in-service unit is reporting and within its limit. The check runs every fifteen minutes; nobody has to look at the page for the monitoring to be happening.
A grey line, *…is out of service since…*	Shown for each storage unit that has been switched off and placed out of service, with the date and the reason recorded. That unit's readings are not judged and it raises no alert; no food may be stored in it. (The walk-in freezer, since 2026-10-12.)
A red line, *N alerts awaiting a recorded response*, and the **open alerts** table	One row per open alert: unit, kind (Out of range, No data, Low battery), the worst value seen, when it opened, and the response. The same alerts are emailed to the unit's recipients, re-sent every 24 hours until acknowledged, and appear as cards on the Notifications page.

What you see	What it is and what it does
Acknowledge button on an open alert	Opens *Acknowledge alert* with one box, **What did you do?**, and a **Record** button. Write what you found and what you did, with the time and the probe reading. **This sentence is the corrective-action record SOP-401 Part 4 requires. It is written once and cannot be edited afterwards**, and the alert shows it from then on. Acknowledging does not close the alert; it closes itself when the readings are back within the limit (or the unit is taken out of service). If product is in doubt, place it on Hold under FSQM-018 and say so in the sentence. Any team account can acknowledge.
Limits button (top right of the card) — *admin and owner only*	Opens one editor per sensor under the heading *Unit limits*; press it again (*Hide limits*) to put them away. Staff accounts see the same limits as a read-only table instead. The fields are in the next table.
Recently closed	The last ten closed alerts: when each opened and closed, and how it ended — the acknowledgement that was recorded, *Not acknowledged* in amber if none was, or *Closed by the system: unit placed out of service* when the unit was switched off while the alert was open. This table and the open one are what the monthly review reads.

The unit limits editor (Limits button; admin and owner)

Field	What it does
In service switch	Storage units only; the Bakery Floor sensor has no switch because it is never judged. On: the unit's readings are judged against its limit every fifteen minutes. Off: the unit is out of service — readings are not judged, no alert is raised, and any alert still open for it is closed at the next check. **Moving the switch opens a box, Reason for the change, and the save refuses without it**: the reason and the date are recorded against the unit (SOP-401 Part 7, "recording the reason") and carried onto that month's FRM-401. The procedure gives this decision to the SQF Practitioner.
Min °F / **Max °F**	The limit. A reading exactly at the limit passes; beyond it fails. An in-service storage unit must have a Max, or the save is refused — a unit with no upper limit would be "monitored" by a check that never fires. The limits in force are stated in SOP-401 Part 2 (refrigerator at or below 41 °F; freezer, while in service, at or below 10 °F); a change here takes effect for all later readings and is not applied backwards.
No-data after (h)	How many hours of silence before a `no_data` alert opens. Default 6. Sensors report roughly hourly, and short gaps are normal.
Notify emails	Who is emailed for this unit, comma-separated. **Blank means every admin and owner account**, which drifts as roles change — set it explicitly (see *Configuring limits and recipients* above).
Notes	Free text shown beside the unit in the read-only table — what the unit holds, where the sensor sits.
Save	Writes the row. Nothing changes until it is pressed; a changed switch with no reason, or an in-service unit with no Max, is refused with a message rather than saved.

Placing a unit out of service (a unit is being switched off)

1. Make sure the unit holds no food. Nothing may be stored in an out-of-service unit (SOP-401 Part 7).
2. On the page, press **Limits**, find the unit, and turn **In service** off. Write the reason in the box that appears (e.g. "Switched off 2026-10-12 — holds no product since the vegan burger line was discontinued") and press **Save**. The decision belongs to the SQF Practitioner; an admin doing it on her say-so records that in the reason.
3. If the unit has an alert open — a unit that has been switched off will usually have an *Out of range* alert within a couple of hours — **acknowledge it** with what was done ("Unit switched off deliberately on …, holds no product, placed out of service"). That is still the response record for that alert; the closing is separate and automatic.

What then happens, without anyone doing anything: within fifteen minutes the job closes the unit's open alerts with the reason *Unit placed out of service* (no "Resolved" email), resolves their cards on the Notifications page, and stops judging the unit. The

24-hour re-send stops with it. The sensor keeps reporting — it runs on its own battery — so the unit's readings go on appearing in the summary and the chart at whatever temperature the switched-off box is; they are not a monitored temperature and the review treats them as such. The grey *out of service* line appears on the Temperature Alerts card. When that month's FRM-401 is filled from the logs, the change appears in *Units placed in or out of service this month* with its date and reason, and the unit's *State this month* reads *Changed during the month* (or *Out of service* in later months, with its temperatures left blank). On FRM-705 the unit's sensor row is marked *Not checked this month — reason recorded* while it is off.

Returning a unit to service

SOP-401 Part 7 sets four conditions before any food goes in: the unit has been cleaned and sanitized; it has been inspected and found in good repair, with door seals intact; its sensor is reporting; and it has held at or below its limit continuously for twenty-four hours. The page supports that order:

1. Switch the unit on and let it pull down. Watch it on the Summary and the chart (set From to the day it was switched on) — the readings are arriving whether or not the unit is in service.
2. Once it is holding at or below its limit, press **Limits**, turn **In service** on, write the reason (e.g. "Frozen ingredients ordered again; cleaned and inspected 2026-11-03") and **Save**. From the next check the unit is judged and alerted on. The amber note under the switch repeats the four conditions.
3. Place no food in it until it has held its limit for twenty-four hours. Record the return on that month's FRM-401: the fill from the logs writes the *Returned to service* row with its date and reason; the reviewer ticks *Return conditions met* and enters *Food first placed in unit* — those two are never filled by the app, because they are the SQF Practitioner's attestation.

Where an alert reaches people

- **Email**, to the unit's recipients, when the alert opens; again every 24 hours while it is unacknowledged; and a short "Resolved" email when it closes because the readings recovered (not when the unit was taken out of service).
- **The Notifications page** (Home → Notifications), as a red-edged card with the alert's wording and a link *Open temperature alerts to acknowledge and record what was done*. Temperature cards have **no Clear button** on purpose: clearing one would make the badge go away without the corrective-action sentence ever being written. The card resolves itself once the alert is acknowledged or closed — at the next twice-daily notifications run (about 07:00 and 15:00 Eastern), or within fifteen minutes when the unit was placed out of service. The open count feeds the Notifications badge in the sidebar, and admins see it in the Today page's Attention card.

The monthly review (FRM-401)

Once a month the SQF Practitioner reviews the month on FRM-401 and signs it (SOP-401 Part 6). The form is filled from the logs in either of two places, and both write the same figures:

- **Start FRM-401 Review** on this page: pick the month (the current month is month-to-date, a past month is the whole month), press **Start review**, and the entry opens with the figures in. Each person can have one open FRM-401 draft; if yours is already open the figures are filled into its blanks, and a draft for a *different* month is reported rather than overwritten.
- **Populate from temperature logs**, the card at the top of an FRM-401 entry that was opened from the SOPs Library instead. Same month picker, same fill, with an Undo; it fills blanks only and refuses a month different from the entry's *Month reviewed*.

What is filled: the month and today's date; each unit's *State this month*, Min, Max and Avg °F (temperatures are left blank for a unit out of service all month); one row per alert raised in the month with its kind, date, worst value, whether it was acknowledged and the action recorded; and one row per unit placed in or out of service in the month, with the date and reason. **What is never filled:** whether each unit held its limit, gaps in logging and why, whether each action was appropriate, the FRM-705 calibration confirmation (the entry shows each sensor's last FRM-705 check under *Device accuracy* to help), manual readings taken while a sensor was down, the return-to-service conditions, findings, the CAPA reference and the signature. Those are the review.

CRITICAL RESET & RESTART CHEAT SHEET

Use these operational diagnostics if readings stall in Supabase, after a server reboot, or when credentials change. **Restarting the service fixes most problems**, because it forces a fresh YoLink token.

Linux System Action	Terminal Command to Execute
Restart the service (apply changes / fix lockups)	`sudo systemctl restart yolink-logger.service`
Check active stream status (is it green/running?)	`sudo systemctl status yolink-logger.service`
View live execution logs (debug packet arrivals)	`sudo journalctl -u yolink-logger.service -f`
Stop the daemon (perform code maintenance)	`sudo systemctl stop yolink-logger.service`
Reload background configs (if modifying service files)	`sudo systemctl daemon-reload`

Standard Troubleshooting Protocols

A. MQTT "Not authorized" / readings stop after ~2 hours

Symptom: the logs show `Connected to YoLink MQTT Broker with result code Not authorized`, and rows stop landing in `temperature_logs` — often after the service has been up for a couple of hours.

Cause: a YoLink access token is valid for only **2 hours (7200s)** and is used as the MQTT username. If the client reconnects using the *same* cached token after it has expired, the broker rejects it as "Not authorized" on every retry.

Fix (already in main.py): the daemon proactively fetches a **fresh token and rebuilds the connection every 100 minutes** (`TOKEN_REFRESH_SECONDS`), staying under the 2-hour expiry, and refuses to "subscribe" on a failed connection. If you see this again: first **restart the service** (forces a new token). If it recurs *immediately* after a restart — not hours later — the credentials are stale; re-issue them (item B) and update `.env` .

B. **YoLink Credential (UAID / Secret) Problems** A token response of `{"state":"error","msg":"client_id not existed"}` means the UAID sent was blank or unrecognized. Re-issue credentials in the YoLink app: **Settings → Account → Advanced Settings → User Access Credentials** (re-issuing invalidates the previous pair), then update `YOLINK_UAID` and `YOLINK_SECRET_KEY` in `.env` and restart. Verify a token can be obtained (run on the VPS with the values loaded):

```
curl -s -X POST https://api.yosmart.com/open/yolink/token \
  -d "grant_type=client_credentials&client_id=$YOLINK_UAID&client_secret=$YOLINK_SECRET_KEY"
```

A JSON body containing `access_token` = credentials good; an error body = env/credential problem. Note: `.env` is read by the *service*, not your login shell — `echo $YOLINK_UAID` in a plain terminal looks empty even when the service is configured correctly.

C. **Database Authentication Failures (401 Errors)** Verify that the secret `service_role` API key remains populated inside `.env` (`SUPABASE_KEY`). The `anon` key is blocked by Row-Level-Security and will fail every insert; it is never safe to expose.

D. **Sensors Not Updating** YoLink equipment transmits selectively on temperature change or on a 30–60 min heartbeat to conserve battery, so gaps are normal. Press the physical **SET** button on the sensor to force an instant broadcast. If one sensor is persistently silent, check its battery gauge on the Temperature Monitoring report — a low battery (level 1) can stop transmissions.
Since alerting was added: short gaps of 30–60 min remain normal and raise nothing, but a silence longer than the unit's `stale_hours` (default 6) now opens a `no_data` alert and emails the recipients. A `no_data` alert means the unit is *unmonitored*, not that it is warm — take manual probe readings until logging resumes.

E. **Verifying Raw Transmission Packets** Stop the daemon and run the script by hand inside the virtualenv to watch raw JSON hit the pipeline live; restart the service when finished:

```
sudo systemctl stop yolink-logger.service
cd /root/yolink-logger && source venv/bin/activate && python main.py
# Ctrl-C when done, then:
sudo systemctl start yolink-logger.service
```

F. **No alert emails arriving** when readings are clearly out of limits. Work down this list in order — each step tells you which half failed.

```
-- (a) did the job run at all?
select status, start_time, return_message from cron.job_run_details d
  join cron.job j using (jobid)
 where j.jobname = 'temperature-alert-check'
 order by start_time desc limit 5;

-- (b) did it open an alert?
select equipment_name, kind, opened_at, worst_value, cleared_at, notify_count
  from public.temperature_alerts order by opened_at desc limit 5;

-- (c) did it try to send, and what happened?
select status, recipient_email, error_message, created_at
  from public.email_send_log
 where template_name = 'temperature-alert' order by created_at desc limit 5;

-- (d) is the unit even eligible? only kind='storage' AND in_service are checked
select equipment_name, kind, in_service, min_f, max_f, stale_hours, notify_emails
  from public.temperature_limits order by equipment_name;
```

A row in (c) reading `not_configured` means either `RESEND_API_KEY` is missing from the Edge Function secrets, or the unit has no recipients *and* no admin/owner account has an email on file. If (d) shows the unit missing entirely, see the join-key warning under *Hardware & Database Mapping*. The function's own response reports `checked` and `skipped` counts, which is the quickest way to confirm coverage.

G. **An alert is stuck open** Alerts close on their own once readings are back inside the limit — confirm the unit is genuinely within range and that readings are still arriving. An alert on a unit that has been *placed out of service* closes at the next run with the reason “Unit placed out of service”; if it has not within fifteen minutes, the `temperature-alert` function has not been redeployed since 2026-10-12 (`npx supabase functions deploy temperature-alert`). A `no_data` alert necessarily stays open while the sensor is silent, because a sensor that is not reporting cannot demonstrate recovery. Alerts are **never** closed by editing the table: it has no UPDATE policy at all, and acknowledging an alert is not the same as closing it.

First-Response Quick Guide

If you see...	Do this first
Report data looks stale / no recent readings	`systemctl status`, then restart the service; watch `journalctl -f`
Logs say “Not authorized”	Restart service (new token). If instant on restart, re-issue YoLink credentials.
Logs say insert / 401 / RLS error	Check `SUPABASE_KEY` is the service-role key in `.env`
One sensor missing, others fine	Press **SET** on that sensor; check its battery level on the report
After a server reboot	Service auto-starts (`Restart=always`); confirm with `systemctl status`

If you see...	Do this first
A `no_data` alert email arrived	The unit is **unmonitored**, not necessarily warm. Probe it by hand and record the reading, then fix the sensor or the daemon
An `out_of_range` alert arrived	Check the door and airflow, probe the unit, correct the cause, then **acknowledge the alert with what you did** — that is the record
Out of limits, but no email came	Troubleshooting item F: cron ran? alert opened? `email_send_log`? unit in service?
Alerts for a unit stopped entirely	Check its `temperature_limits` row still exists and its `equipment_name` matches `DEVICE_MAPPING` exactly — and that the unit is **In service** (the grey line on the Temperature Alerts card says if it is not)
A unit is being switched off (or on again)	Limits → In service switch, with the reason; see *Placing a unit out of service* and *Returning a unit to service* above